



Tilo - 文艺生活记录APP 产品设计文档

一、项目概述

1.1 项目背景

在快节奏的现代生活中，人们需要一个简单而美好方式来记录日常。市场上的记录类APP大多功能复杂、界面沉闷，缺乏美感。Tilo 希望通过梦幻清新的设计，让记录成为一种享受。

1.2 项目目标

- 打造一款视觉优先的生活记录APP
- 通过AI降低记录门槛，让用户轻松生成优质内容
- 提供流畅的云端同步体验

1.3 核心价值

维度	描述
用户价值	轻松记录、美好呈现、AI辅助
商业价值	云存储增值、AI功能付费点
技术价值	现代化架构、大厂技术栈实践

二、设计流程

2.1 需求来源

原始需求：做一个记录日常的APP，参考小红书风格，瀑布流展示，图片为主。

用户痛点分析：

- 现有APP功能太复杂，记录门槛高
- 界面设计缺乏美感和个性
- 需要手动输入大量文字

2.2 竞品分析

竞品	优点	缺点
小红书	内容质量高、瀑布流展示	社区属性强、不够私密
日记APP	隐私保护好	界面陈旧、缺乏美感
备忘录	轻量便捷	无图片支持、无云同步

差异化机会：轻量 + 美观 + AI辅助

2.3 设计决策

设计理念：Less is More

决策项	选择	理由
界面风格	梦幻粉紫配色	差异化、清新少女风
布局方式	瀑布流	图片优先、浏览体验好
内容生成	AI辅助	降低记录门槛
数据存储	云端优先	多设备同步、安全

三、思考链路

3.1 核心问题拆解

代码块

- 1
- 如何让用户愿意记录？
- 2
- ↓
- 3
- 记录门槛太高怎么办？
- 4
- ↓ 用AI生成降低门槛
- 5
- 输入什么让用户有表达欲？
- 6
- ↓ 图片优先、标题可见
- 7
- 如何让界面更好看？
- 8
- ↓ 梦幻配色、Material Design 3
- 9
- 如何保证数据安全？
- 10
- ↓ Supabase云端存储

3.2 功能优先级排序

优先级	功能	理由
P0	记录添加+展示	核心闭环
P0	图片上传	核心差异化
P1	AI内容生成	降低门槛
P1	标签筛选	整理功能
P2	搜索功能	查找需求
P2	GPS定位	体验优化

3.3 技术选型思考

为什么用 Jetpack Compose?

- 声明式UI，代码简洁
- 预览功能强大，开发效率高
- Google主推，趋势技术

为什么用 Supabase?

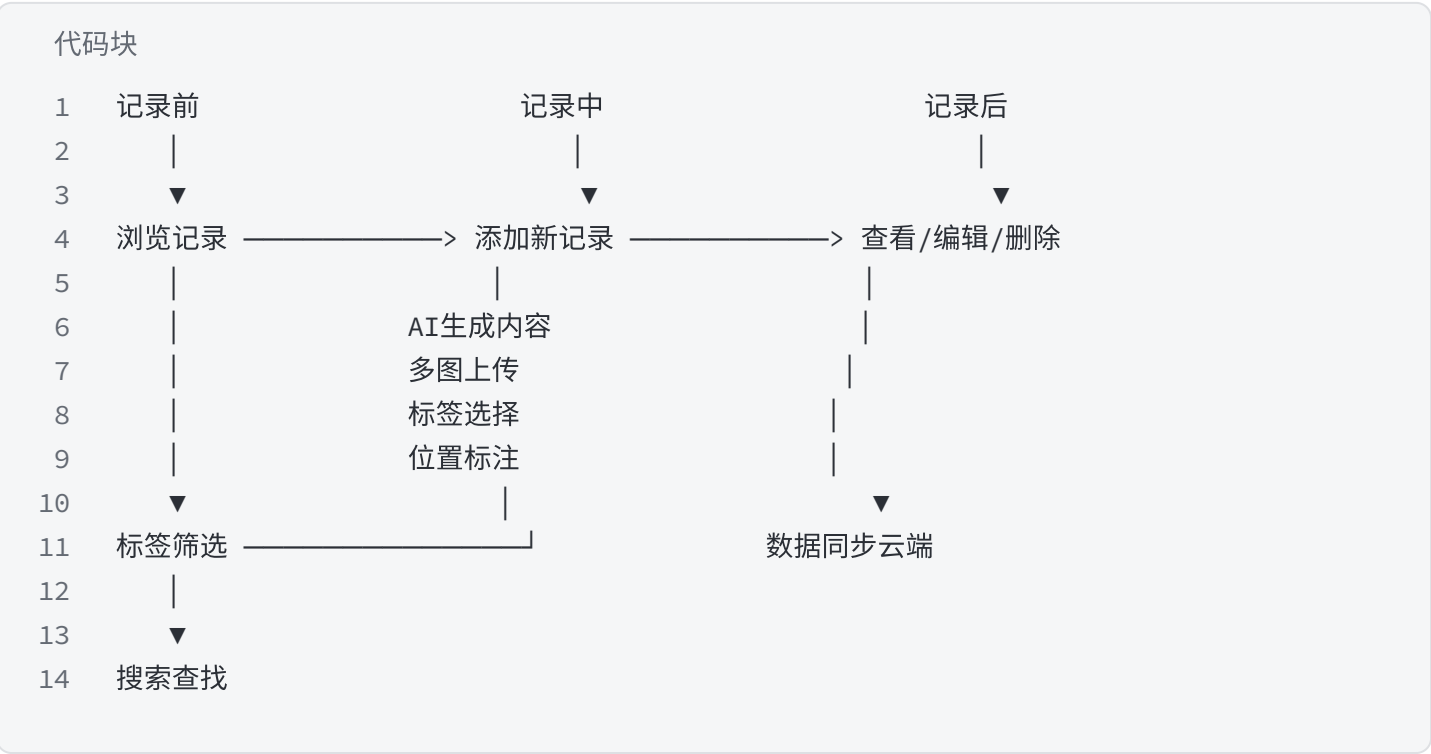
- 开箱即用的后端服务
- 免费额度够用
- REST API简单易集成

为什么用 ModelScope?

- 国内可用，无需翻墙
- 千问模型效果好
- API定价合理

四、需求拆解

4.1 用户故事地图



4.2 功能模块拆解

4.2.1 记录模块

功能点	输入	处理	输出
创建记录	内容+图片	保存到 Supabase	返回成功
查看记录	记录ID	获取详情	展示页面
编辑记录	新内容	更新数据库	返回成功
删除记录	记录ID	删除数据	返回列表

4.2.2 图片模块

功能点	输入	处理	输出
选择图片	相册选择	Uri获取	图片列表
上传图片	图片数据	上传到Storage	公开URL
删除图片	图片索引	从列表移除	更新显示

4.2.3 AI模块

功能点	输入	处理	输出
生成内容	关键词	调用千问API	标题+内容+标签
本地备选	关键词	本地生成	基础内容

4.2.4 位置模块

功能点	输入	处理	输出
GPS定位	权限	获取坐标	经纬度
地址解析	坐标	Geocoder	地址文字

4.3 数据模型设计

代码块

```
1  data class Record(  
2      id: Int?,           // Supabase自增ID  
3      title: String?,     // 标题（AI生成或手动）  
4      content: String,    // 内容正文  
5      location: String?,  // 位置描述  
6      latitude: Double?,  // 纬度  
7      longitude: Double?, // 经度  
8      tag: String?,       // 标签  
9      imageUrl: String?,  // 图片URL（逗号分隔）  
10     timestamp: String   // 时间戳  
11 )
```

4.4 页面结构

代码块

```
1 MainActivity
2   |—— ListScreen (主页)
3   |   |—— TopBar (搜索)
4   |   |—— FilterChips (标签筛选)
5   |   |—— StaggeredGrid (瀑布流列表)
6   |—— DetailScreen (详情)
7   |   |—— Images (多图展示)
8   |   |—— Content (内容)
9   |   |—— Actions (编辑/删除)
10  |—— AddEditScreen (添加/编辑)
11     |—— TitleInput (标题)
12     |—— ContentInput (内容)
13     |—— AiGenerate (AI生成)
14     |—— LocationInput (位置)
15     |—— TagSelector (标签)
16     |—— ImagePicker (图片)
```

五、技术实现亮点

5.1 架构设计

代码块

```
1 UI Layer (Composable)
2   |
3   ▼
4 ViewModel (StateFlow)
5   |
6   ▼
7 Repository (数据抽象)
8   |
9   ▼
10 Data Sources (Supabase API)
```

5.2 关键代码片段

Flow数据流:

代码块

```
1 fun getAllRecords(): Flow<List<Record>> = flow {
2     val response = api.getAllRecords().execute()
3     if (response.isSuccessful) {
4         emit(response.body() ?: emptyList())
5     }
6 }.flowOn(Dispatchers.IO)
```

瀑布流布局：

代码块

```
1 LazyVerticalStaggeredGrid(
2     columns = StaggeredGridCells.Fixed(2),
3     verticalItemSpacing = 10.dp,
4     horizontalArrangement = Arrangement.spacedBy(10.dp)
5 )
```

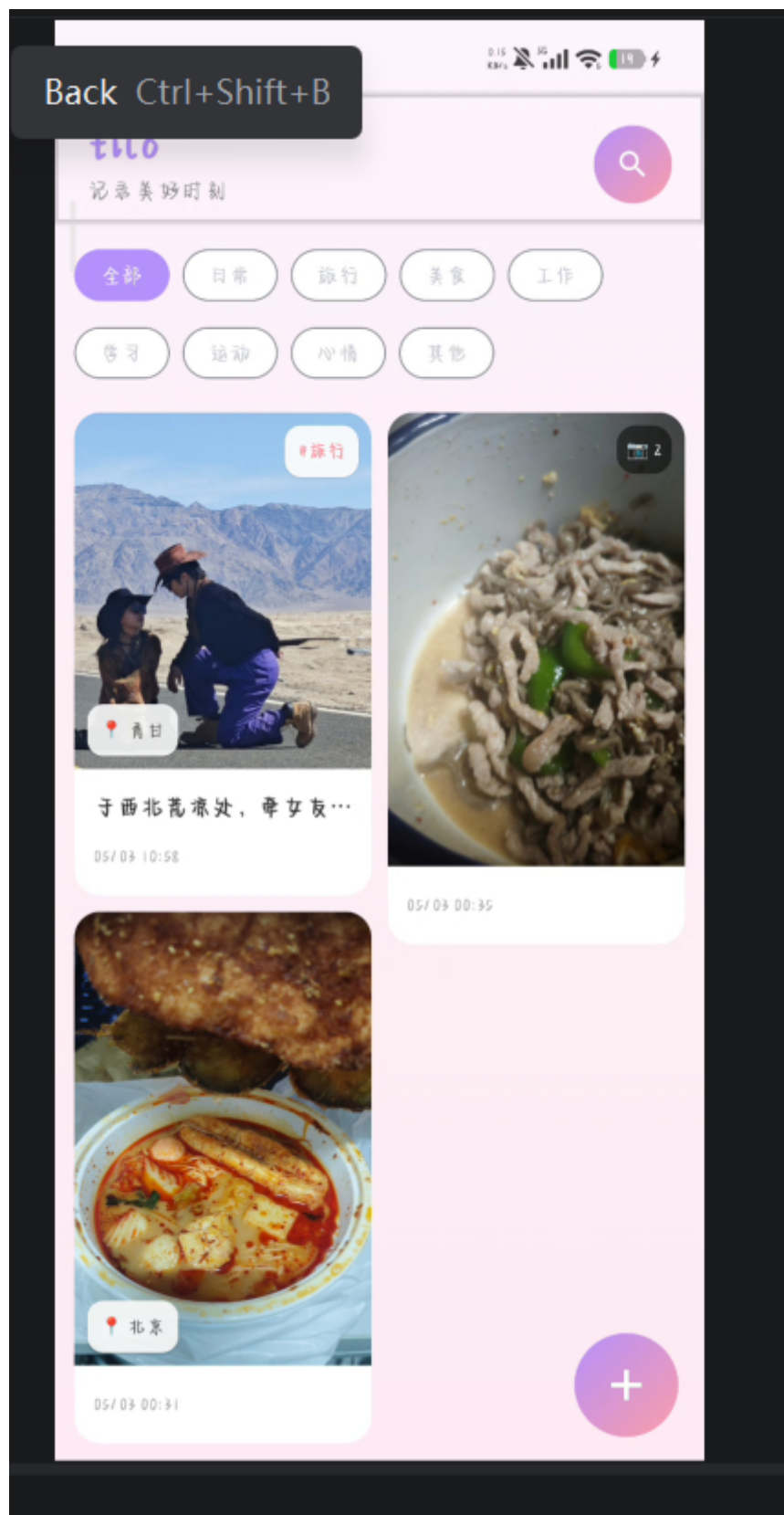
5.3 性能优化

- 图片使用 Coil 异步加载和缓存
- Supabase REST API 减少请求量
- Coroutines 非阻塞异步处理

六、截图展示

6.1 主界面瀑布流

截图位置：screenshots/1-主界面瀑布流.png



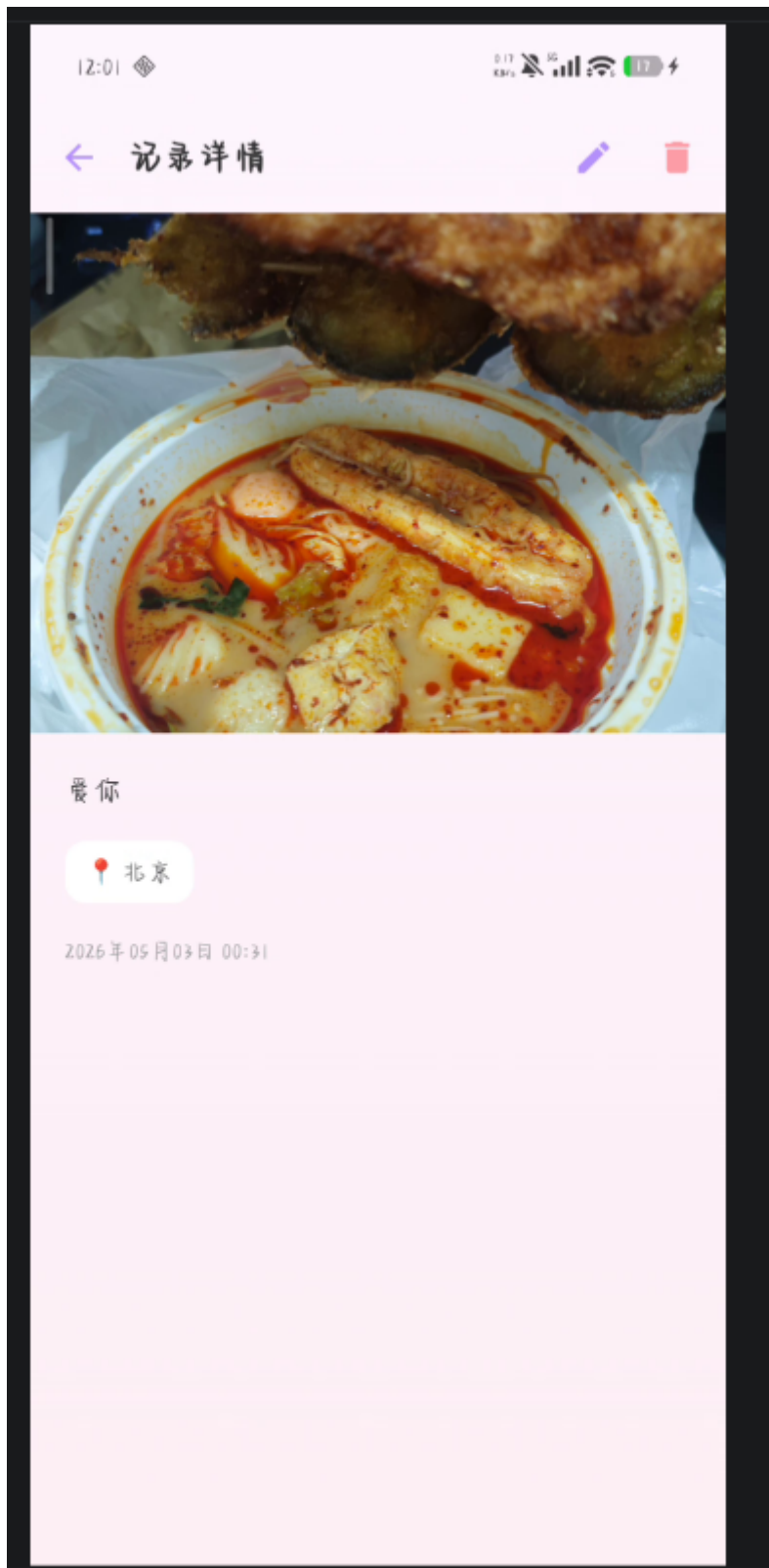
6.2 AI生成功能

截图位置：screenshots/2-AI生成功能.png



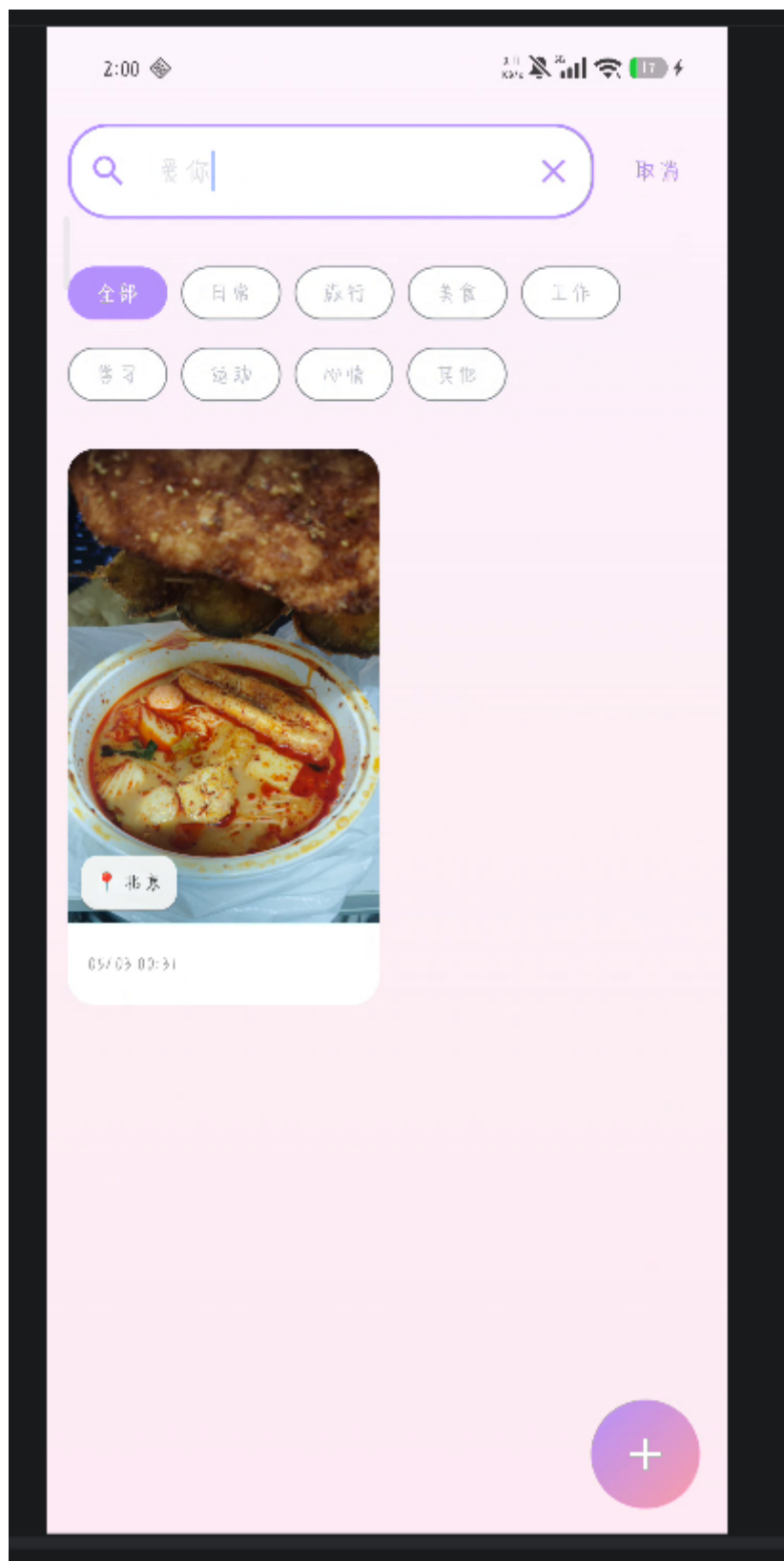
6.3 记录详情页

截图位置: screenshots/3-记录详情.png



6.4 标签筛选

截图位置：screenshots/4-标签筛选.png



七、总结与反思

7.1 项目收获

维度	收获
技术	掌握了Jetpack Compose、Supabase集成、AI API调用
产品	学会了需求拆解、功能优先级排序
设计	理解了Material Design 3设计理念

7.2 改进空间

- 添加深色模式支持
- 优化AI生成的文艺风格
- 增加数据导出功能
- 添加心情天气等额外维度

7.3 未来规划

1. 短期：完善细节，优化AI生成效果
2. 中期：添加社交分享功能
3. 长期：探索AI记忆功能，让记录更智能

文档更新时间：2026年4月